

DCGAN 生成漫画头像 实验手册



华为技术有限公司

版权所有 © 华为技术有限公司 2024。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



和其他华为商标均为华为技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

华为技术有限公司

地址： 深圳市龙岗区坂田华为总部办公楼 邮编：518129

网址： <http://e.huawei.com>



目录

1 实验介绍	2
1.1 实验目的	2
1.2 实验清单	2
1.3 开发平台介绍	2
1.4 开发环境搭建	3
2 DCGAN 生成漫画头像	4
2.1 DCGAN 网络介绍背景知识	4
2.1.1 GAN 基础原理	4
2.1.2 DCGAN	5
2.2 数据下载与处理	5
2.3 创建网络	8
2.4 损失函数和优化器	11
2.5 模型训练	11
2.6 结果展示	13
2.7 实验总结	15

1 实验介绍

生成式对抗网络是一种深度学习模型，是近年来复杂分布上无监督学习最具前景的方法之一。本实验通过 MindSpore 框架搭建 DCGAN 网络，并使用动漫头像数据集来训练此生成式对抗网络，接着使用该网络生成动漫头像图片。

1.1 实验目的

本案例使用 MindSpore 框架进行深度学习网络 DCGAN 搭建，利用动漫头像数据集进行模型训练、图像生成。通过本实验可以了解到深度学习任务开发的具体流程，包括：数据处理、创建网络、连接网络和损失函数 损失函数和优化器和模型训练及图像生成；了解经典神经网络 DCGAN 的结构特点和创新设计，以及深度学习网络搭建的重要概念。

1.2 实验清单

实验	简述	难度	软件环境	开发环境
DCGAN生成漫画头像	本实验使用MindSpore框架DCGAN网络，利用动漫头像数据集进行模型训练和图像生成	中级	MindSpore2.2	ModelArts、Ascend

1.3 开发平台介绍

昇腾 (Ascend) 是华为自研的一款基于达芬奇架构的 AI 处理器，具有超高算力的和极高的能效比，最高可达 320TFLOPS (FP16) 的浮点计算能力。昇腾的片上系统 (SoC) 还集成了多个 CPU、DVPP 和任务调度器，因而具有自我管理能力和，可以充分发挥其高算力的优势。强大的矩阵、向量运行能力，在进行海量数据运算的神经网络训练场景有巨大的优势。

昇思 MindSpore (官方网站 : <https://www.mindspore.cn/>) 是一种适用于端边云场景的新型开源深度学习训练/推理框架。 MindSpore 提供了友好的设计和高效的执行,旨在提升数据科学家和算法工程师的开发体验,并为 Ascend AI 处理器提供原生支持,以及软硬件协同优化。同时, MindSpore 作为全球 AI 开源社区,致力于进一步开发和丰富 AI 软硬件应用生态。

ModelArts (官方网站 : <https://console.huaweicloud.com/modelarts/>) 是面向 AI 开发者的一站式开发平台,提供海量数据预处理及半自动化标注、大规模分布式训练、自动化模型生成及模型按需部署能力,帮助用户快速创建和部署 AI 应用,管理全周期 AI 工作流。

1.4 开发环境搭建

本实验开发环境为华为云 ModelArts 平台,环境搭建方式请参考如下手册:



华为云ModelArts
环境搭建手册.docx

2 DCGAN 生成漫画头像

2.1 DCGAN 网络介绍背景知识

2.1.1 GAN 基础原理

最初，GAN 由 Ian Goodfellow 于 2014 年发明，并在论文 Generative Adversarial Nets 中首次进行了描述，GAN 由两个不同的模型组成：生成器和判别器：

- 生成器的任务是生成看起来像训练图像的“假”图像；
- 判别器需要判断从生成器输出的图像是真实的训练图像还是虚假的图像。

在训练过程中，生成器会不断尝试通过生成更好的假图像来骗过判别器，而判别器在这过程中也会逐步提升判别能力。这种博弈的平衡点是，当生成器生成的假图像和训练数据图像的分布完全一致时，判别器拥有 50% 的真假判断置信度。

下面，我们首先定义一些在整个过程中需要用到的符号：

参数	参数说明
x	代表图像数据。
$D(x)$	判别器网络，给出图像判定为真实图像的概率。

表2-1 判别器参数表

由于我们在判别过程中需要处理图像，因此要为 $D(x)$ 提供 CHW 格式且大小为 $3 \times 64 \times 64$ 的图像。当 x 来自训练数据时， $D(x)$ 数值应该趋近于 1，而当 x 来自生成器时， $D(x)$ 数值应该趋近于 0。因此 $D(x)$ 也可以被认为是传统的二分类器。

接下来我们来定义生成器的表示方法：

参数	参数说明
z	标准正态分布中提取出的隐向量。

$G(z)$	表示将隐向量 z 映射到数据空间的生成器函数。
--------	---------------------------

表2-2 生成器参数表

函数 $G(z)$ 的目标是将一个随机高斯噪声 z 通过一个生成网络生成一个和真实数据分布 $p_{data}(x)$ 差不多的数据分布，其中 θ 是网络参数，我们希望找到 θ 使得 $p_{G(x;\theta)}$ 和 $p_{data}(x)$ 尽可能的接近。

$D(G(z))$ 是生成器 G 生成的假图像被判定为真实图像的概率。

如 Goodfellow 的论文中所述， D 和 G 在进行一场博弈， D 想要最大程度的正确分类真图像与假图像，也就是参数 $\log D(x)$ ；而 G 试图欺骗 D 来最小化假图像被识别到的概率，也就是参数 $\log(1-D(G(z)))$ 。GAN 的损失函数为：

$$\min_G \max_D V(D, G) = E_{x \sim p_{data}(x)} [\log(D(x))] + E_{z \sim P_Z(z)} [\log(1 - D(G(z)))]$$

从理论上讲，此博弈游戏的平衡点是 $p_{G(x;\theta)} = p_{data}(x)$ ，此时判别器会随机猜测输入是真图像还是假图像。然而，GAN 的收敛可行性仍在研究当中，在实际场景中模型并不会被训练到这一步。

2.1.2 DCGAN

DCGAN（深度卷积对抗生成网络，Deep Convolutional Generative Adversarial Networks）是 GAN 的直接扩展。不同之处在于，DCGAN 会分别在判别器和生成器中使用卷积和卷积转置层。

它最早由 Radford 等人在论文 Unsupervised Representation Learning With Deep Convolutional Generative Adversarial Networks 中进行描述。判别器由分层的卷积层、BatchNorm 层和 LeakyReLU 激活层组成。输入是 $3 \times 64 \times 64$ 的图像，输出是该图像为真图像的概率。生成器则是由转置卷积层、BatchNorm 层和 ReLU 激活层组成。输入是标准正态分布中提取出的隐向量 z ，输出是 $3 \times 64 \times 64$ 的 RGB 图像。

本教程将使用 70,171 张动漫头像数据集来训练一个生成对抗网络，接着使用该网络生成动漫头像图片。

2.2 数据下载与处理

步骤 1 下载实验所需模块

开始实验之前，需要首先更新环境中的依赖包，在 Notebook 中输入以下命令并运行。

```
# 安装过程可能会有异常提示，不影响实验操作
!pip install download
%env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
```

步骤 2 数据下载

首先我们将数据集下载到指定目录下并解压。

示例代码如下：

```
from download import download
url = "https://download.mindspore.cn/dataset/Faces/faces.zip"
path = download(url, "./faces", kind="zip", replace=True)
```

下载后的数据集目录结构如下：

```
faces/
├── faces
│   ├── 0.jpg
│   ├── 1.jpg
│   ├── 2.jpg
│   ├── 3.jpg
│   ├── 4.jpg
│   ├── ...
│   ├── 70169.jpg
│   └── 70170.jpg
```

步骤 3 数据处理相关的一些输入定义：

参数	参数说明
batch_size	训练中使用的批量大小，DCGAN论文使用的批量大小为128；
image_size	训练图像的大小，此实现默认为`64x64`，如果需要其他尺寸，则必须同时更改`D`和`G`的结构；
nc	输入图像中的彩色通道数，因为此次是彩色图像所以设为3；
nz	隐向量的长度；
ngf	设置通过生成器的特征图的深度；
ndf	设置通过判别器传播的特征图的深度；

num_epochs	要运行的训练周期数，训练更长的时间可能会导致更好的结果，但也会花费更长的时间；
lr	训练的学习率；
beta1	Adam优化器的`beta1`超参数。如DCGAN论文所述，该数字应为0.5；

表2-3 模型参数说明

```

batch_size = 128      # 批量大小
image_size = 64       # 训练图像空间大小
nc = 3                # 图像彩色通道数
nz = 100              # 隐向量的长度
ngf = 64              # 特征图在生成器中的大小
ndf = 64              # 特征图在判别器中的大小
num_epochs = 10       # 训练周期数
lr = 0.0002           # 学习率
beta1 = 0.5           # Adam 优化器的 beta1 超参数

```

步骤 4 定义 create_dataset_imagenet`函数对数据进行处理和增强操作。

```

import numpy as np
import mindspore.dataset as ds
import mindspore.dataset.vision as vision

def create_dataset_imagenet(dataset_path):
    """数据加载"""
    dataset = ds.ImageFolderDataset(dataset_path,
                                    num_parallel_workers=4,
                                    shuffle=True,
                                    decode=True)

    # 数据增强操作
    transforms = [
        vision.Resize(image_size),
        vision.CenterCrop(image_size),
        vision.HWC2CHW(),
        lambda x: (x / 255).astype("float32")
    ]

    # 数据映射操作
    dataset = dataset.project('image')

```

```
dataset = dataset.map(transforms, 'image')

# 批量操作
dataset = dataset.batch(batch_size)
return dataset

dataset = create_dataset_imagenet('./faces')
```

步骤 5 通过 create_dict_iterator 函数将数据转换成字典迭代器，然后使用`matplotlib`模块可视化部分训练数据。

```
import matplotlib.pyplot as plt

def plot_data(data):
    # 可视化部分训练数据
    plt.figure(figsize=(10, 3), dpi=140)
    for i, image in enumerate(data[0][:30], 1):
        plt.subplot(3, 10, i)
        plt.axis("off")
        plt.imshow(image.transpose(1, 2, 0))
    plt.show()

sample_data = next(dataset.create_tuple_iterator(output_numpy=True))
plot_data(sample_data)
```

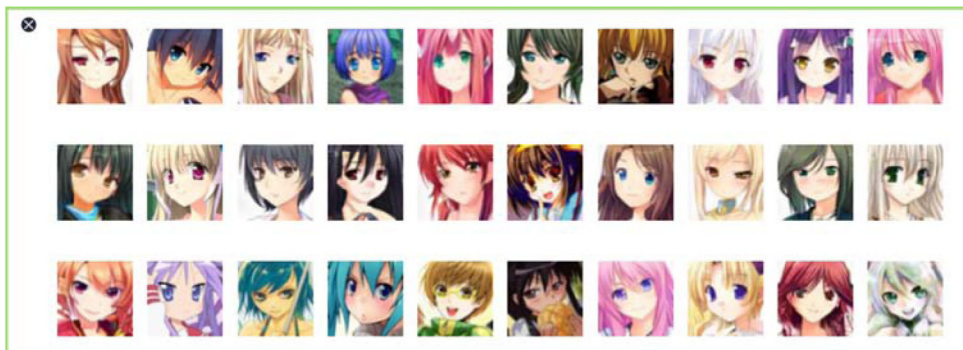


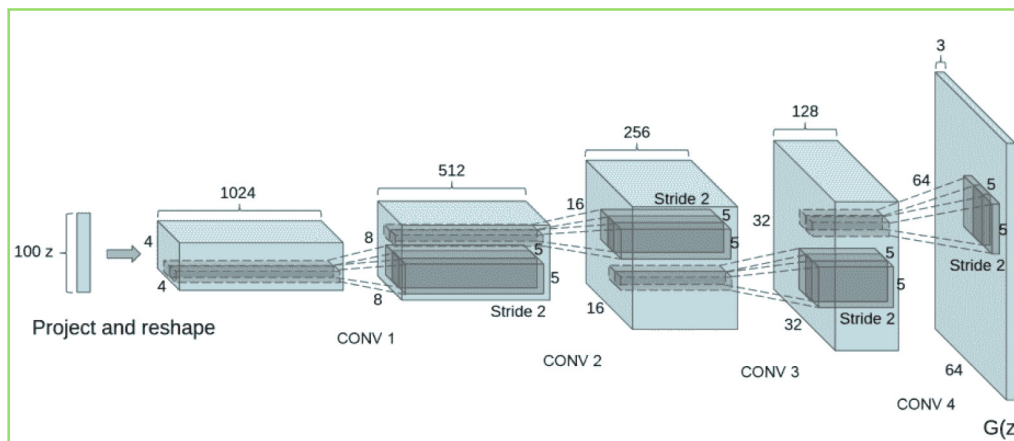
图2-1 训练数据图

2.3 创建网络

当处理完数据后，就可以来进行网络的搭建了。按照 DCGAN 论文中的描述，所有模型权重均应从 mean 为 0，sigma 为 0.02 的正态分布中随机初始化。

步骤 1 生成器

DCGAN 论文生成图像如下所示：



以下是生成器的代码实现：

```
import mindspore as ms
from mindspore import nn, ops
from mindspore.common.initializer import Normal

weight_init = Normal(mean=0, sigma=0.02)
gamma_init = Normal(mean=1, sigma=0.02)

class Generator(nn.Cell):
    """DCGAN 网络生成器"""

    def __init__(self):
        super(Generator, self).__init__()
        self.generator = nn.SequentialCell([
            nn.Conv2dTranspose(nz, ngf * 8, 4, 1, 'valid', weight_init=weight_init),
            nn.BatchNorm2d(ngf * 8, gamma_init=gamma_init),
            nn.ReLU(),
            nn.Conv2dTranspose(ngf * 8, ngf * 4, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf * 4, gamma_init=gamma_init),
            nn.ReLU(),
            nn.Conv2dTranspose(ngf * 4, ngf * 2, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf * 2, gamma_init=gamma_init),
            nn.ReLU(),
            nn.Conv2dTranspose(ngf * 2, ngf, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf, gamma_init=gamma_init),
```



```

        nn.ReLU(),
        nn.Conv2dTranspose(ngf, nc, 4, 2, 'pad', 1, weight_init=weight_init),
        nn.Tanh()
    )

    def construct(self, x):
        return self.generator(x)

generator = Generator()

```

步骤 2 判别器

如前所述，判别器 D 是一个二分类网络模型，输出判定该图像为真实图的概率。通过一系列的 Conv2d、BatchNorm2d 和 LeakyReLU 层对其进行处理，最后通过 Sigmoid 激活函数得到最终概率。

DCGAN 论文提到，使用卷积而不是通过池化来进行下采样是一个好方法，因为它可以让网络学习自己的池化特征。

判别器的代码实现如下：

```

class Discriminator(nn.Cell):
    """DCGAN 网络判别器"""

    def __init__(self):
        super(Discriminator, self).__init__()
        self.discriminator = nn.SequentialCell(
            nn.Conv2d(nc, ndf, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.LeakyReLU(0.2),
            nn.Conv2d(ndf, ndf * 2, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf * 2, gamma_init=gamma_init),
            nn.LeakyReLU(0.2),
            nn.Conv2d(ndf * 2, ndf * 4, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf * 4, gamma_init=gamma_init),
            nn.LeakyReLU(0.2),
            nn.Conv2d(ndf * 4, ndf * 8, 4, 2, 'pad', 1, weight_init=weight_init),
            nn.BatchNorm2d(ngf * 8, gamma_init=gamma_init),
            nn.LeakyReLU(0.2),
            nn.Conv2d(ndf * 8, 1, 4, 1, 'valid', weight_init=weight_init),
        )
        self.adv_layer = nn.Sigmoid()

    def construct(self, x):
        out = self.discriminator(x)
        out = out.reshape(out.shape[0], -1)
        return self.adv_layer(out)

discriminator = Discriminator()

```


2.4 损失函数和优化器

当定义了 D 和 G 后，接下来将使用 MindSpore 中定义的二进制交叉熵损失函数 BCELoss，为 D 和 G 加上损失函数和优化器。

```
# 定义损失函数
adversarial_loss = nn.BCELoss(reduction='mean')
```

这里设置了两个单独的优化器，一个用于 D，另一个用于 G。这两个都是 lr = 0.0002 和 beta1 = 0.5 的 Adam 优化器。

```
# 为生成器和判别器设置优化器
optimizer_D = nn.Adam(discriminator.trainable_params(), learning_rate=lr, beta1=beta1)
optimizer_G = nn.Adam(generator.trainable_params(), learning_rate=lr, beta1=beta1)
optimizer_G.update_parameters_name('optim_g.')
optimizer_D.update_parameters_name('optim_d.')
```

2.5 模型训练

训练分为两个主要部分：训练判别器和训练生成器。

- 训练判别器

训练判别器的目的是最大程度地提高判别图像真伪的概率。按照 Goodfellow 的方法，是希望通过提高其随机梯度来更新判别器，所以我们要最大化 $\log D(x) + \log(1 - D(G(z)))$ 的值。

- 训练生成器

如 DCGAN 论文所述，我们希望通过最小化 $\log(1 - D(G(z)))$ 来训练生成器，以产生更好的虚假图像。

在这两个部分中，分别获取训练过程中的损失，并在每个周期结束时进行统计，将 fixed_noise 批量推送到生成器中，以直观地跟踪 G 的训练进度。

步骤 1 下面实现模型训练正向逻辑：

```
def generator_forward(real_imgs, valid):
    # 将噪声采样为发生器的输入
    z = ops.standard_normal((real_imgs.shape[0], nz, 1, 1))

    # 生成一批图像
    gen_imgs = generator(z)

    # 损失衡量发生器绕过判别器的能力
    g_loss = adversarial_loss(discriminator(gen_imgs), valid)
```

```

return g_loss, gen_imgs

def discriminator_forward(real_imgs, gen_imgs, valid, fake):
    # 衡量鉴别器从生成的样本中对真实样本进行分类的能力
    real_loss = adversarial_loss(discriminator(real_imgs), valid)
    fake_loss = adversarial_loss(discriminator(gen_imgs), fake)
    d_loss = (real_loss + fake_loss) / 2
    return d_loss

grad_generator_fn = ms.value_and_grad(generator_forward, None,
                                       optimizer_G.parameters,
                                       has_aux=True)
grad_discriminator_fn = ms.value_and_grad(discriminator_forward, None,
                                          optimizer_D.parameters)

@ms.jit
def train_step(imgs):
    valid = ops.ones((imgs.shape[0], 1), mindspore.float32)
    fake = ops.zeros((imgs.shape[0], 1), mindspore.float32)

    (g_loss, gen_imgs), g_grads = grad_generator_fn(imgs, valid)
    optimizer_G(g_grads)
    d_loss, d_grads = grad_discriminator_fn(imgs, gen_imgs, valid, fake)
    optimizer_D(d_grads)

    return g_loss, d_loss, gen_imgs

```

步骤 2 循环训练网络，每经过 50 次迭代，就收集生成器和判别器的损失，以便于后面绘制训练过程中损失函数的图像。

```

import mindspore

G_losses = []
D_losses = []
image_list = []

total = dataset.get_dataset_size()
for epoch in range(num_epochs):
    generator.set_train()
    discriminator.set_train()
    # 为每轮训练读入数据
    for i, (imgs, ) in enumerate(dataset.create_tuple_iterator()):
        g_loss, d_loss, gen_imgs = train_step(imgs)
        if i % 100 == 0 or i == total - 1:
            # 输出训练记录
            print('[%2d/%d][%3d/%d]   Loss_D:%7.4f   Loss_G:%7.4f' % (
                epoch + 1, num_epochs, i + 1, total, d_loss.asnumpy(), g_loss.asnumpy()))

```

```
D_losses.append(d_loss.asnumpy())
G_losses.append(g_loss.asnumpy())

# 每个 epoch 结束后，使用生成器生成一组图片
generator.set_train(False)
fixed_noise = ops.standard_normal((batch_size, nz, 1, 1))
img = generator(fixed_noise)
image_list.append(img.transpose(0, 2, 3, 1).asnumpy())

# 保存网络模型参数为 ckpt 文件
mindspore.save_checkpoint(generator, "./generator.ckpt")
mindspore.save_checkpoint(discriminator, "./discriminator.ckpt")
```

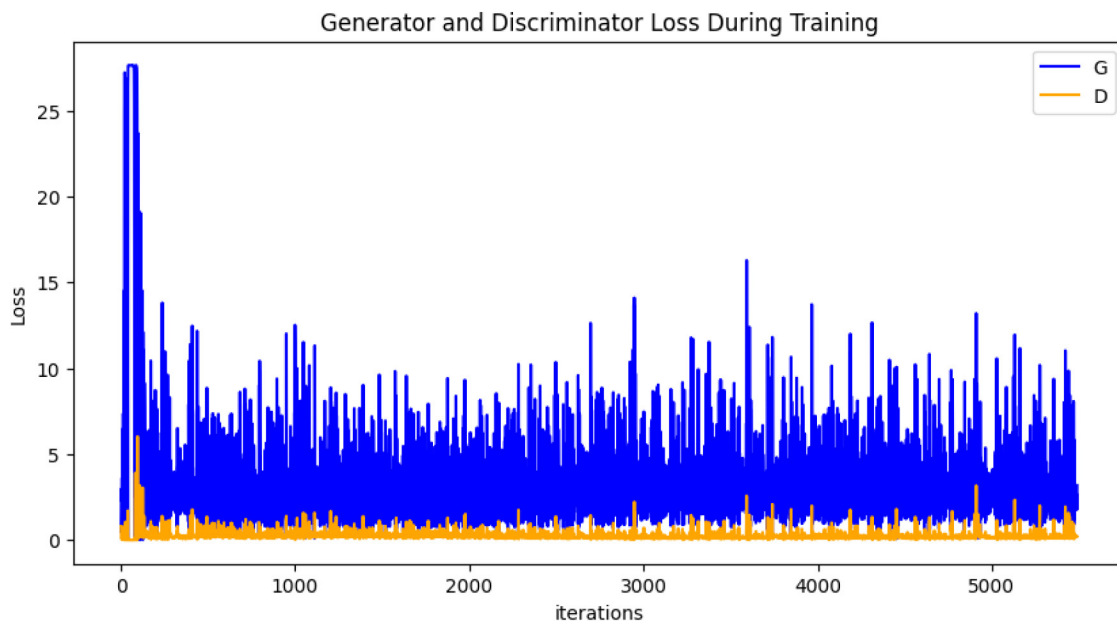
完整的 10 个 epoch 模型训练结果，如下图：

```
[1/10][ 1/549]   Loss_D: 0.8013   Loss_G: 0.5065
[1/10][101/549]   Loss_D: 0.1116   Loss_G: 13.0030
[1/10][201/549]   Loss_D: 0.1037   Loss_G: 2.5631
...
[1/10][401/549]   Loss_D: 0.6240   Loss_G: 0.5548
[1/10][501/549]   Loss_D: 0.3345   Loss_G: 1.6001
[1/10][549/549]   Loss_D: 0.4250   Loss_G: 1.1978
...
[10/10][501/549]   Loss_D: 0.2898   Loss_G: 1.5352
[10/10][549/549]   Loss_D: 0.2120   Loss_G: 3.1816
```

2.6 结果展示

步骤 1 运行下面代码，描绘‘D’和‘G’损失与训练迭代的关系图：

```
plt.figure(figsize=(10, 5))
plt.title("Generator and Discriminator Loss During Training")
plt.plot(G_losses, label="G", color='blue')
plt.plot(D_losses, label="D", color='orange')
plt.xlabel("iterations")
plt.ylabel("Loss")
plt.legend()
plt.show()
```



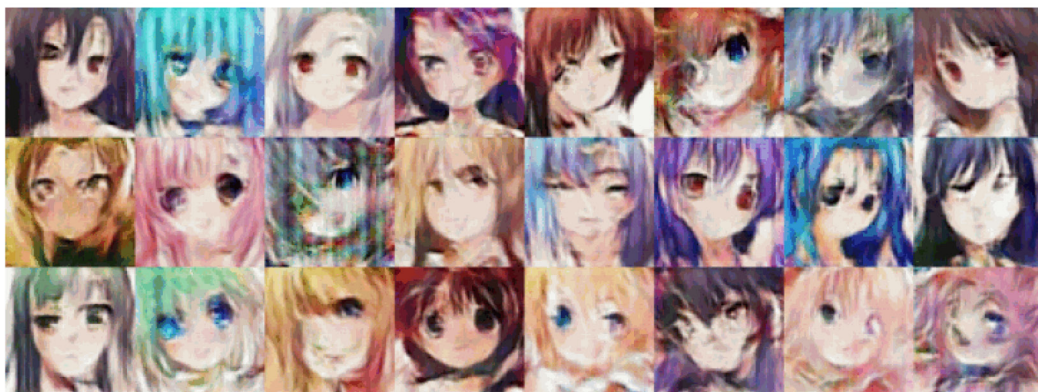
步骤 2 可视化训练过程中通过隐向量`fixed\_noise`生成的图像。

```
import matplotlib.pyplot as plt
import matplotlib.animation as animation

def showGif(image_list):
    show_list = []
    fig = plt.figure(figsize=(8, 3), dpi=120)
    for epoch in range(len(image_list)):
        images = []
        for i in range(3):
            row = np.concatenate((image_list[epoch][i * 8:(i + 1) * 8]), axis=1)
            images.append(row)
        img = np.clip(np.concatenate((images[:]), axis=0), 0, 1)
        plt.axis("off")
        show_list.append([plt.imshow(img)])

    ani = animation.ArtistAnimation(fig, show_list, interval=1000, repeat_delay=1000, blit=True)
    ani.save('./dcgan.gif', writer='pillow', fps=1)

showGif(image_list)
```



从上面的图像可以看出，随着训练次数的增多，图像质量也越来越好。如果增大训练周期数，当 num_epochs 达到 50 以上时，生成的动漫头像图片与数据集中的较为相似，下面我们通过加载生成器网络模型参数文件来生成图像，代码如下：

```
# 从文件中获取模型参数并加载到网络中
mindspore.load_checkpoint("./generator.ckpt", generator)
fixed_noise = ops.standard_normal((batch_size, nz, 1, 1))
img64 = generator(fixed_noise).transpose(0, 2, 3, 1).asnumpy()
fig = plt.figure(figsize=(8, 3), dpi=120)
images = []
for i in range(3):
    images.append(np.concatenate((img64[i * 8:(i + 1) * 8]), axis=1))
img = np.clip(np.concatenate((images[:]), axis=0), 0, 1)
plt.axis("off")
plt.imshow(img)
plt.show()
```



2.7 实验总结

本章实验在华为云 ModelArts 平台上使用 MindSpore 完整地实现了一个基于 DCGAN 的生成式对抗网络模型，主要是利用动漫头像数据集进行模型训练和图像生成，通过实验使学员了解生成式对抗网络的算法模型，数据处理，模型训练等知识内容。